

SECURITY TAXONOMY OF LARGE LANGUAGE MODEL (LLM) VULNERABILITIES

Lalitha Sravanti Dasu (UCD ID: 925576376)

Ketki Kulkarni (UCD ID: 924332825)

TABLE OF CONTENTS

1. INTRODUCTION	2
2. A FOUR-PART TAXONOMY OF LLM VULNERABILITIES	2
JAILBREAKING	2
Main Approaches	2
Key Findings	3
Advantages of the Jailbreak research	4
Limitations	4
PROMPT INJECTION	4
High-Level Themes Across the Literature	5
Detailed Analysis of the Prompt Injection Category	5
Advantages of Current Research	6
Limitations	6
DATA POISONING	7
Main Approaches	7
Key Findings	8
Advantages of Data Poisoning	8
Limitations	9
HALLUCINATION	9
High-Level Themes	9
Detailed Analysis of the Hallucination Category	10
Advantages of the Current Hallucination Research	11
Limitations	12
3. A UNIFIED FRAMEWORK: ANALYZING THE INTERCONNECTIONS	12
4. CONCLUSION AND FUTURE DIRECTIONS	17
5. CONTRIBUTIONS	17
6. REVIEWED LITERATURE	18

1. INTRODUCTION

As LLMs like GPT and Claude become standard in critical applications, their security vulnerabilities are becoming a major concern. Unlike traditional software, these models learn from data patterns, making them susceptible to unique forms of manipulation like leaking sensitive info and generating harmful content.

Currently, research on these security issues is fragmented. This report aims to provide a holistic view of the threat landscape by organizing the research into four primary categories:

- **Jailbreaking:** Tricking a model into bypassing its own safety filters.
- **Prompt Injection:** Hiding malicious instructions within the input to deceive the model.
- **Data Poisoning:** Corrupting the model's behavior by manipulating its training data.
- **Hallucinations:** When a model confidently generates false or nonsensical information.

The goal of this paper is to consolidate scattered information into a single, structured framework. By analyzing how these vulnerabilities are defined, executed, and defended against, I aim to provide a clear guide to understanding the current state of LLM security.

2. A FOUR-PART TAXONOMY OF LLM VULNERABILITIES

JAILBREAKING

Jailbreaking refers to techniques used to trick a language model at inference time so that it ignores safety rules or content filters. Unlike data poisoning, jailbreak attacks do not require any access to training data. Instead, attackers craft input prompts that exploit weaknesses in the model's instruction-following behavior. Examples include asking the model to role-play a fictional character, providing multi-step instructions, or using adversarial phrasing that confuses safety mechanisms. The key advantage of studying jailbreaks is that it directly evaluates models as deployed, revealing practical vulnerabilities. The challenge is that success depends heavily on the exact wording, context, or sequence of prompts, making universal defenses very difficult.

Main Approaches

a) Adversarial Prompting

Papers: [PAIR](#); [Adversarial Jailbreaks Across Closed Models](#)

This approach focuses on crafting carefully worded prompts that make the model output content it normally wouldn't. Experiments in PAIR and the work by Zou et al. show that even highly instruction-tuned models can be tricked with a relatively small set of prompts. The advantage is that it exposes fundamental weaknesses in the model's understanding of instructions. The downside is that minor changes in phrasing can make the attack fail, so it's not completely reliable across all scenarios.

b) Role-Playing or Context Framing

Papers: [“Do Anything Now”](#); [TAP](#)

Here, harmful instructions are embedded in a story, hypothetical scenario, or fictional context. For example, asking the model to “pretend you are a hacker” or “write as a criminal mastermind” can bypass safety filters. Experiments show that models often follow these instructions even when the requested content would normally be restricted. The advantage is that it highlights how models blindly follow context, but the limitation is the dependence on precise framing; small changes in context can block the attack.

c) Multi-Step or Chained Prompts

Papers: [TAP](#); [JailbreakBench](#)

Attackers can break instructions into sequences where each step appears harmless individually but cumulatively bypasses safety rules. Experiments test success rates across prompt sequences, showing which steps are critical. This method reflects realistic user interactions where multiple turns occur. Its limitation is complexity both for attackers and defenders because there are countless possible sequences.

d) Large-Scale Benchmarking and Evaluation

Papers: [JailbreakBench](#); [SELFDEFEND](#)

Some studies focus on systematically measuring jailbreak success across multiple models and prompts. JailbreakBench creates a benchmark for testing LLM robustness, while SELFDEFEND proposes defenses where models actively identify and block malicious inputs. The advantage is that these approaches bring rigor and comparability, but they are limited by the evolving nature of attacks; benchmarks can quickly become outdated.

Key Findings

Papers: [“Do Anything Now”: Characterizing and Evaluating In-The-Wild Jailbreak Prompts on LLMs](#); [JailbreakBench](#); [PAIR](#); [TAP](#); [Adversarial Jailbreaks Across Closed Models](#); [SELFDEFEND](#)

- Jailbreaking works even against top-tier instruction-tuned models.
- Larger models are sometimes more vulnerable, as they generalize patterns faster and follow instructions more literally.

- Some jailbreak strategies transfer across models and versions, indicating systemic weaknesses.
- Existing safety mechanisms like RLHF or content filters reduce but don't fully eliminate jailbreak success.
- Effective attacks usually depend on precise phrasing, context, and multi-step logic.

Advantages of the Jailbreak research

- Directly tests deployed models – No need to access training data; shows real-world behavior.
- Exposes alignment gaps – Highlights where models fail to follow safety rules.
- Identifies systemic weaknesses across models – Certain attacks work across multiple LLMs.
- Guides safety improvements – Insights help improve RLHF, moderation filters, and defense strategies.
- Quick feedback – Experiments can be conducted in real time without retraining.
- Supports benchmarking – Benchmarks like JailbreakBench provide scientific ways to measure robustness.
- Encourages proactive safety research – Helps anticipate future threats and design defenses in advance.

Limitations

Papers: [“Do Anything Now”](#); [TAP](#); [PAIR](#); [Adversarial Jailbreaks Across Closed Models](#)

- Highly dependent on phrasing and context; small changes can invalidate attacks.
- Infinite prompt variations make it impossible to cover all possibilities.
- Ethical concerns limit real-world testing of truly harmful content.
- Vulnerability varies a lot with the model's size and architecture, making universal defenses difficult. Also, benchmarks get outdated quickly as new attack methods appear.

PROMPT INJECTION

Prompt injection refers to attacks where malicious text is inserted into a prompt or hidden in external data to trick an LLM. This manipulation makes the model ignore its original instructions and produce harmful or uncontrolled outputs. Across the papers I surveyed, the main goals are to define this threat, test how strong current models are against it, or suggest defenses like structured querying and detection. Together, these works help build

a better understanding of why LLMs are vulnerable and show the limits of our current defenses.

High-Level Themes Across the Literature

- Prompt injection is now widely recognized as a core flaw in the architecture of LLMs.
- Most attacks work because LLMs cannot effectively distinguish between "instructions" and "data."
- Benchmarks are now testing robustness more systematically, and they show that even fine-tuned models fail when faced with subtle attacks.
- Defenses usually fall into two categories: changing the structure of the prompt (like using SQL-style interfaces) or trying to detect bad content before the model sees it.
- None of the defenses are perfect. Every paper demonstrates that attackers can still bypass these mechanisms.

Detailed Analysis of the Prompt Injection Category

Formalization and Benchmarking

Papers: [Formalizing & Benchmarking PI](#), [SEP Dataset](#), [Open-Prompt-Injection](#)

These works establish prompt injection as a clear and measurable security problem. By defining specific attack types, they allow us to evaluate models in a more scientific way. For example, SEP shows that even innocent-looking text can override system prompts, which proves there is a structural weakness in how LLMs parse instructions. Open-Prompt-Injection adds to this by testing real-world scenarios and multi-modal conditions. The main advantage of these papers is that they bring rigor to the field. However, their biggest limitation is generalization because benchmarks become outdated very quickly as attackers invent new methods.

Defensive Structural Approaches

Papers: [Defending with Structured Queries](#)

The structured-query approach argues that we shouldn't just try to make LLMs "more obedient." Instead, we should limit what they can do with natural language. By separating the user's input from the executable actions, we reduce the model's ability to misinterpret instructions. This method is very precise, especially for enterprise or database uses. The downside is that it is quite narrow and doesn't work well for open-ended creative tasks. Also, these defenses rely on developers strictly enforcing interfaces, which people often skip for the sake of convenience.

Detection-Based Defenses

Papers: [Deployable Detection for Prompt Injection](#)

Detection systems try to catch harmful text before it ever reaches the model. They are cheap to run and easy to deploy on any model. Detection works well for obvious jailbreak attempts, but it struggles when the attack is subtle or blended into the sentence. The fundamental limitation here is that if the LLMs themselves cannot tell instructions apart from content, detection systems built on similar technology will face the same confusion. These defenses are reactive rather than preventative.

Indirect Prompt Injection

Papers: [TopicAttack](#)

TopicAttack is interesting because it shows that attackers don't even need to give direct commands. Simply changing the topic of conversation can cause the model to drift away from its safety restrictions. This exposes a cognitive vulnerability where LLMs are too dependent on context. It breaks the assumption that prompt injection requires explicit "hack" words. The limitation of studying this is that these attacks are inconsistent, which makes them hard to benchmark and hard to defend against.

Advantages of Current Research

- It establishes formal definitions and benchmarks so we can reproduce evaluations.
- It covers both direct attacks and indirect vectors to give a full view of the threats.
- It introduces practical tools we can use now, such as detectors and guardrails.
- It identifies the root weaknesses in LLM architecture to guide future designs.
- It proves that robustness is a system design problem, not just a training issue.
- It highlights real-world use cases, making the research applicable to deployed apps.

Limitations

- No defense offers a complete solution since all are just partial fixes.
- Benchmarks cannot keep up with the speed of attacker creativity.
- LLMs still lack a hard boundary between "task instruction" and "input content."
- Many evaluations rely on static prompts and miss dynamic conversational attacks.
- Detection approaches are easy to bypass by paraphrasing or hiding the text.
- Structured-query defenses are too restrictive for creative or open-ended tasks.
- There are no standard metrics, so it is hard to compare robustness claims.
- Real-world attacks vary by application, so universal defenses are difficult to build.
- Indirect attacks like topic transitions are still poorly understood.

- Most works focus on the application layer instead of fixing the model architecture itself.

DATA POISONING

Data poisoning refers to attacks where training data is maliciously manipulated so that a model learns undesirable behavior. This happens during training not inference unlike jailbreaks. Attackers can hide triggers or bad examples in the datasets, which causes the model to output unsafe results later when it sees specific inputs. The core issue is the model silently learns bad habits. This is a major threat to safety, but detecting the poisoned data is tough because the malicious changes are so subtle within huge datasets.

Main Approaches

a) Instruction Backdoors

Papers: [Backdoor Vulnerabilities of Instruction Tuning for Large LMs](#) ; [Learning to Poison LLMs During Instruction Tuning](#)

This approach embeds backdoor instructions into training data so that the model responds to a specific trigger in a predictable way. Experiments usually measure how reliably these triggers produce harmful outputs. The advantage is that attackers can control model behavior systematically. The limitation is that effective backdoors often require access to training data and can sometimes be detected if carefully monitored.

b) Continuous Prompt Backdoors

Papers: [BadPrompt: Backdoor Attacks on Continuous Prompts](#)

Here, attackers manipulate learned embeddings or continuous prompts rather than discrete examples. Experiments show that even small modifications in continuous prompt space can produce significant unintended behavior. The advantage is that this method can bypass defenses based on filtering discrete data. Its limitation is that it requires technical access to the model or fine-tuning process, so it's harder to deploy in black-box settings.

c) Scaling Trends in Poisoning

Papers: [Jailbreak-Tuning: Exploring Scaling Trends in Data Poisoning](#)

This line of work explores how model size affects poisoning effectiveness. Experiments show that larger LLMs can sometimes be more vulnerable to poisoning because they generalize patterns more efficiently. The advantage is that these studies help predict risks

as models scale. Limitations include that the trends are not uniform across all architectures, so generalization is tricky.

d) Persistent Pre-Training Poisoning

Papers: [Persistent Pre-Training Poisoning of LLMs \(Zhang et al., 2024\)](#)

This method injects poisoned data early in pre-training so that harmful behavior persists even after fine-tuning. Experiments show that triggers inserted in pre-training are harder to remove later. The advantage is long-lasting impact, making defenses challenging. The limitation is that early-stage poisoning is resource-intensive and harder for attackers to access in practice.

Key Findings

- Poisoning can persist through fine-tuning, especially if introduced early in training.
- Even small amounts of poisoned data can significantly influence model outputs.
- Triggers can be embedded in instruction datasets, prompts, or continuous embeddings.
- Larger models can sometimes amplify the effects of poisoning due to pattern generalization.
- Detecting poisoned examples is non-trivial, as they often appear benign individually.
- Data poisoning provides a stealthy method to alter model behavior in ways that are difficult to trace or reverse.

Advantages of Data Poisoning

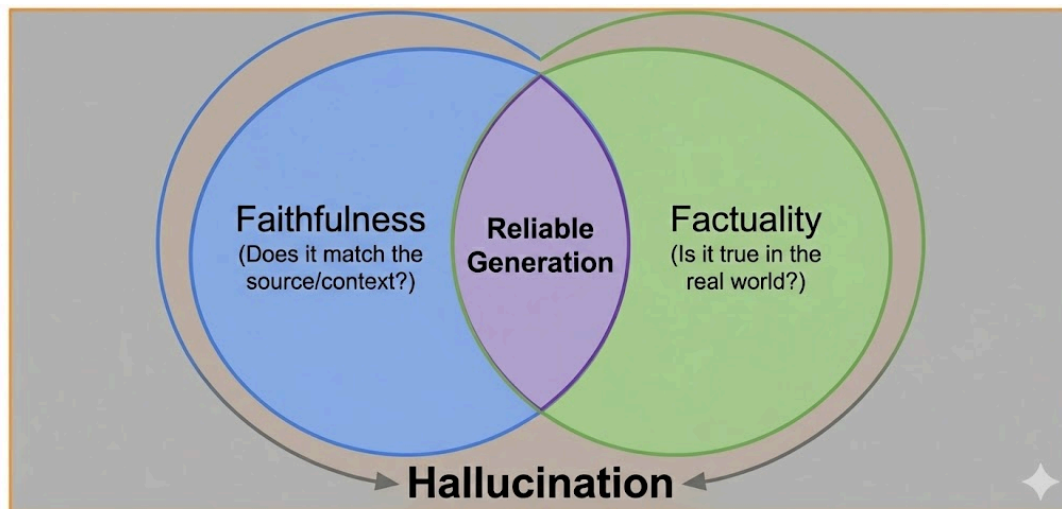
- Systematic influence – Attackers can reliably induce specific behaviors through triggers.
- Stealthy and long-lasting – Malicious effects can persist across fine-tuning and pre-training stages.
- Highlights model vulnerabilities – Shows which model components are sensitive to small data changes.
- Informs defense strategies – Studying poisoning helps design filtering, auditing, and robust training methods.
- Scalable attacks – Even minimal poisoned datasets can affect large models if carefully designed.
- Benchmarking potential – Studies like Jailbreak-Tuning quantify how model size and architecture affect vulnerability.

Limitations

- Requires access to training or fine-tuning datasets, which limits feasibility in some cases.
- Hard to detect poisoned data without extensive auditing.
- Impact depends on model size, architecture, and fine-tuning methods.
- Persistent effects make mitigation resource-intensive, often requiring retraining or prompt filtering.
- Ethical constraints restrict real-world testing of harmful content.
- Some poisoning techniques may be specific to particular datasets or models, limiting generalizability.

HALLUCINATION

Hallucination happens when a language model generates content that looks confident but is actually factually incorrect, ungrounded, or logically invalid. In this category review, I synthesize key ideas from six foundational papers: SelfCheckGPT, Faithful Chain-of-Thought Reasoning, TruthfulQA, Understanding and Mitigating Hallucination in LLMs and LVLMs, Factuality of Large Language Models: A Survey, and Rethinking Factuality Evaluation and Hallucination Metrics.



High-Level Themes

- Hallucination includes factual inaccuracies, fabricated reasoning steps, and confusion between different modalities in Large Vision-Language Models (LVLMs).
- Evaluation remains the core challenge because hallucinations often appear fluent and plausible even when they are wrong.

- Detection and mitigation require a mix of different strategies, including sampling-based checks, grounding, reasoning pipelines, and decoding interventions.
- Larger models reduce some types of hallucinations but can actually amplify others by increasing the model's confidence in incorrect answers.
- Surveys emphasize that hallucination is a systemic issue and cannot be solved by any single technique.

Detailed Analysis of the Hallucination Category

Evaluation and Benchmarking

Papers: [TruthfulQA](#); [Rethinking Factuality Evaluation and Hallucination Metrics](#); [Factuality of LLMs Survey](#)

This group of papers defines how we should measure hallucination. TruthfulQA revealed that models tend to mimic common human misconceptions rather than choosing the correct fact, which motivated the creation of more truth-oriented benchmarks. Later work argues that evaluating factuality must be done across multiple dimensions. We need to capture faithfulness, consistency, and harm severity rather than just looking at surface-level accuracy. The surveys reinforce the idea that reliable measurement is a prerequisite for making real progress, especially for open-ended generation and real-world applications.

Black-Box and Sampling-Based Detection

Papers: [SelfCheckGPT](#); [Understanding and Mitigating Hallucination in LLMs and LVLMS](#)

These works show that hallucinations often correlate with internal uncertainty. SelfCheckGPT tests this by sampling multiple generations and checking whether the outputs agree with each other. Low consistency typically signals a hallucination. This black-box approach is attractive because it requires no access to the model's internal weights and can be deployed on closed-source APIs. Broader survey work places SelfCheckGPT within a family of uncertainty-based detectors, showing that while these methods are effective for detection, they cannot correct hallucinations on their own.

Faithful Reasoning and Chain-of-Thought

Papers: [Faithful Chain-of-Thought Reasoning](#)

Hallucinations also arise from flawed intermediate reasoning. The Faithful Chain-of-Thought line of work separates the process into two stages. First, the model converts the question into a structured reasoning format, and second, a deterministic

solver executes it. This reduces fabricated reasoning steps and makes the intermediate logic easier to check. These papers highlight that hallucination is not just about missing facts. It is also about unreliable cognitive processes inside the model.

Taxonomy, Causes, and Mitigation Strategies

Papers: [Factuality of LLMs Survey](#); [Understanding and Mitigating Hallucination in LLMs and LVLMs](#)

Survey papers bring together decades of understanding about why hallucinations emerge. The causes include training data noise, overconfident decoding, retrieval failures, modality priors, and hallucinated reasoning steps. They also categorize the different layers of mitigation, such as retrieval augmentation, contrastive decoding, calibration, dataset curation, and alignment methods like RLHF. This category provides the conceptual foundation for all hallucination research by revealing that the problem has multiple causes that are all interconnected.

Multimodal and LVLM-Specific Hallucinations

Papers: [Understanding and Mitigating Hallucination in LLMs and LVLMs](#)

For LVLMs, hallucinations occur when the model invents objects, attributes, or spatial relations that do not exist in the image. These papers show that weak cross-modal alignment, noisy captions, and strong text priors can override the visual evidence. They emphasize defenses specific to this modality, such as visual grounding checks and improved alignment losses. This expands the hallucination problem into real-world domains where incorrect visual reasoning can have serious consequences.

Decoding-Level Interventions

Papers: [Understanding and Mitigating Hallucination in LLMs and LVLMs](#); [Factuality Survey](#)

Several papers show that hallucinations can be reduced at inference time. Techniques like contrastive decoding, fact-aware reranking, or confidence-weighted generation help penalize outputs that are internally inconsistent or have low confidence. These approaches are appealing because they do not require retraining and can be layered onto deployed systems. Collectively, they show that decoding strategies play a critical role in controlling hallucinations.

Advantages of the Current Hallucination Research

- It establishes strong foundations for evaluation, such as TruthfulQA and metric rethinking, so hallucinations can be measured meaningfully.

- It produces deployable, black-box detectors like SelfCheckGPT that can work with any model.
- It advances reasoning faithfulness via structured Chain-of-Thought pipelines that reduce fabricated steps.
- It provides comprehensive taxonomies that clarify the causal structure of hallucination.
- It introduces practical decoding interventions that reduce hallucinations without the need for retraining.
- It extends hallucination research into multimodal settings, which increases its real-world relevance.

Limitations

No single benchmark captures all hallucination types, especially conversational or multimodal ones.

- Detectors can identify instability but cannot provide factual corrections.
- Many mitigation methods involve trading off creativity, fluency, or output diversity.
- Structured reasoning approaches require complex engineering and do not generalize to all tasks.
- Data quality issues remain foundational because models trained on noisy corpora will continue to hallucinate.
- LVLM hallucinations lag behind in understanding, and existing mitigation tools are still primitive.

3. A UNIFIED FRAMEWORK: ANALYZING THE INTERCONNECTIONS

While understanding each vulnerability in isolation is a crucial first step, the true novelty of this survey lies in analyzing how these disparate threats connect, overlap, and differ. By synthesizing the findings from the previous sections, we can build a unified framework that reveals the bigger picture of LLM security. This holistic view demonstrates that these are not four separate problems but rather interconnected symptoms of foundational challenges in model architecture, data integrity, and alignment.

Contrast of Attack Timelines and Mechanisms

The four vulnerabilities can be broadly categorized by when they are executed in the model lifecycle: during training or at inference. This distinction dictates the attacker's requirements, the attack's mechanism, and the challenges for detection.

Inference-Time Attacks (Jailbreaking & Prompt Injection)	Training-Time Attacks (Data Poisoning)
Mechanism: Relies on crafting clever inputs (prompts) to manipulate the model's behavior <i>after</i> deployment.	Mechanism: Relies on maliciously manipulating the training or fine-tuning data to embed hidden backdoors <i>before</i> deployment.
Attacker Requirements: Only requires access to the model's input interface (eg. API or chat window).	Attacker Requirements: Requires access to the model's data supply chain, either during pre-training or instruction tuning.
Detection Challenge: Must be detected in real-time. Defences are difficult because of the infinite variations of prompts and the subtlety of attacks like indirect prompt injection.	Detection Challenge: Must be detected before or during training. Defenses are difficult because malicious data points can be subtle and are hidden within massive datasets, requiring extensive auditing.

Attack Stage: Training vs. Inference

- A primary distinction among these vulnerabilities is the stage at which they occur in the model's lifecycle.
- Data Poisoning is exclusively a training-time attack. It requires access to the data pipeline to corrupt the model's foundational knowledge before it is ever deployed.
- Jailbreaking and Prompt Injection are inference-time attacks. They do not alter the model itself but instead manipulate its behavior in real-time by exploiting how it processes input prompts.
- Hallucination is more complex. It can originate from noisy or biased training data (a passive form of data integrity failure) or emerge during inference due to flawed decoding strategies or reasoning failures.

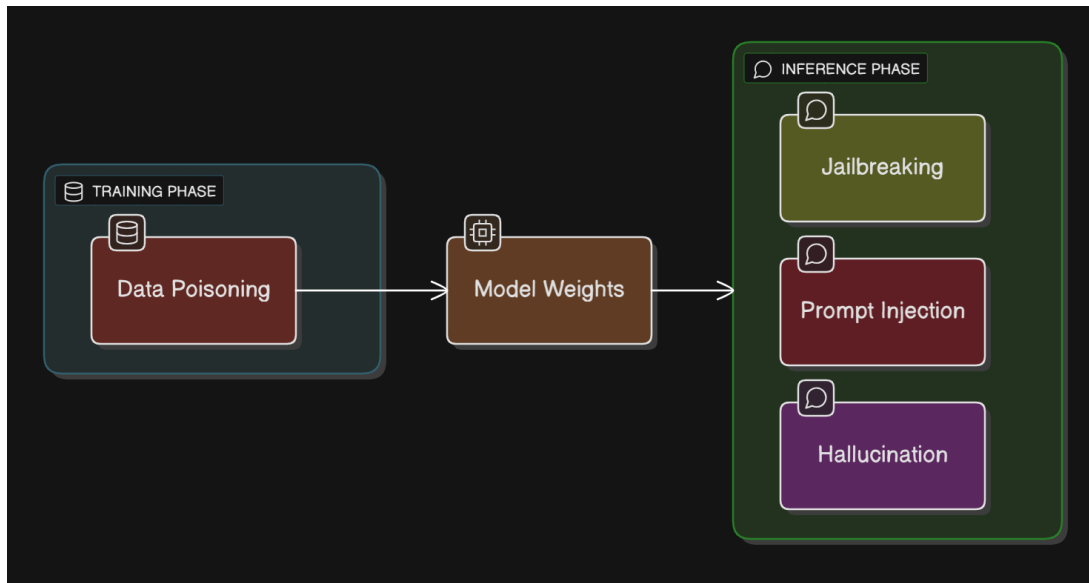


Figure 1: The LLM Security Lifecycle. This diagram illustrates where each vulnerability impacts the model pipeline, distinguishing between training-time attacks (Data Poisoning) and inference-time failures.

Actor Intent: Malicious vs. Inherent

The vulnerabilities can also be distinguished by the intent behind them.

- Jailbreaking, Prompt Injection, and Data Poisoning are fundamentally adversarial, requiring a malicious actor with a clear objective to bypass safety, hijack control, or embed hidden backdoors. Defense strategies for these threats must therefore focus on threat modeling and adversarial robustness.
- Hallucination, in contrast, is typically an inherent model failure. It can occur without any malicious intent from the user and represents a breakdown in the model's ability to reason or recall facts accurately. Defense strategies here focus on improving faithfulness, factuality, and reliability.

Root Cause: Data vs. Logic

Finally, we can analyze the underlying cause of each failure.

- Prompt Injection stems from a core architectural flaw where the model cannot distinguish between data and instructions. It confuses the two, allowing data to be executed as a command.
- Jailbreaking exploits flawed instruction-following logic. The model follows the literal instructions of an adversarial prompt so completely that it overrides its own pre-programmed safety alignment.

- Data Poisoning is a failure of data integrity. The model's knowledge base is corrupted, leading it to learn malicious patterns as if they were legitimate information.
- Hallucination represents a failure of reasoning or grounding. The model either cannot access the correct information or fails to construct a logically sound and factually accurate response based on that information.

The Shared Root Cause of Evasion Attacks

A fundamental connection exists between Jailbreaking and Prompt Injection: both vulnerabilities stem from the same architectural weakness. As identified in the prompt injection literature, this weakness is the LLM's inability to reliably distinguish between trusted system instructions and untrusted user-provided data. This architectural flaw is exploited differently but to the same effect. Jailbreaking, as seen in *TAP's role-playing* scenarios, manipulates the semantic context to make the model prioritize the user's narrative over its system prompt. Prompt injection, as benchmarked by the *SEP Dataset*, manipulates the syntactic structure, using hidden text to hijack the instruction parser. Both achieve the same end: elevating user input to the level of a trusted system command.

Causal Relationships and Threat Amplification

These vulnerabilities do not exist in isolation; they can interact and amplify one another, creating cascading security failures.

- Data Poisoning can create latent backdoors that make a model more susceptible to specific Jailbreak prompts. The concept of *Jailbreak-Tuning* shows how a model can be poisoned during its training phase to respond to certain triggers, effectively pre-programming a vulnerability that a later jailbreak attempt can easily exploit. The poisoning attack primes the model for a future inference-time compromise.
- Hallucination acts as a universal enabling condition for other attacks. A model's inability to maintain coherent factual grounding (hallucination), as measured by benchmarks like *TruthfulQA*, is precisely what makes it susceptible to the fabricated realities presented in jailbreak role-playing and deceptive prompt injection contexts. Lacking foundational reliability, the model is more easily misled into accepting the attacker's fictional premise and complying with the embedded malicious instructions.

Collective Impact on Model Reliability and Trust

While these four vulnerabilities differ in their mechanics ranging from manipulation of training data to runtime exploitation, they converge on the same consequence: they undermine the foundational reliability required for AI adoption. Whether a model is

jailbroken by a clever user or simply hallucinates a fact, the operational result is a breakdown in predictability that erodes user trust.

This shared impact demonstrates that a fragmented, reactive approach to security is no longer sufficient. We cannot simply patch these issues one by one. Instead, securing LLMs requires a holistic strategy that addresses vulnerabilities across the entire lifecycle, from the initial curation of data to real-time monitoring. As these models become more complex, shifting focus from "patching bugs" to proactive "security by design" will be the defining challenge for future research.

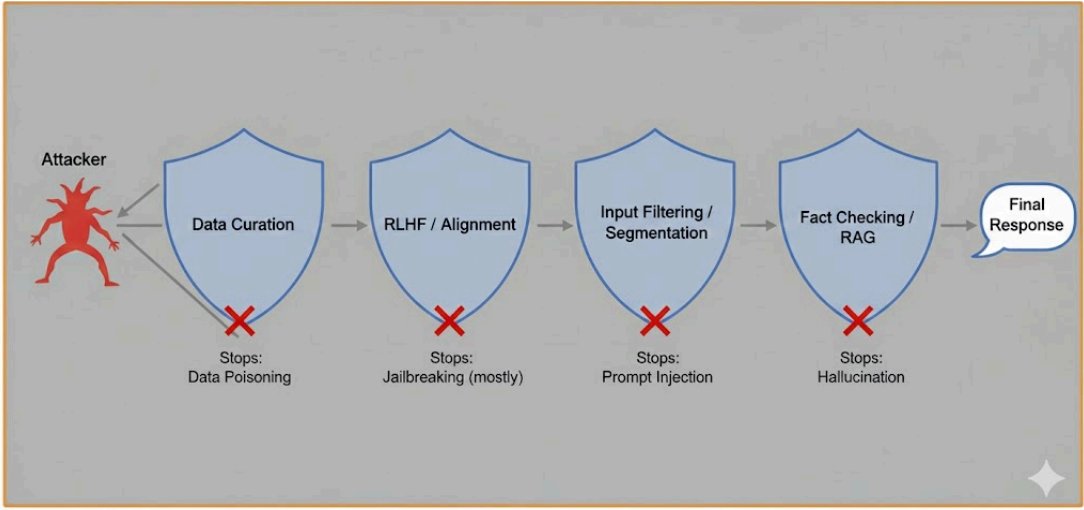


Figure 2: The "Defense-in-Depth" Layered Model

Feature	Data Poisoning	Jailbreaking	Prompt Injection	Hallucination
Lifecycle Stage	Training/Fine Tuning	Inference (Runtime)	Inference (Runtime)	Inference/ Training
Root Cause	Corrupted Training Data	Flawed Safety Filters	No Boundary between Data and Instructions	Probabilistic Nature of the Model
Actor Intent	Malicious (Adversarial)	Malicious (Adversarial)	Malicious (Adversarial)	Unintentional (Model Failure)
Primary Impact	Backdoors/ Biased Behaviour	Generation of Forbidden Content	Unauthorized Actions/ Data Leaks	Misinformation/ Loss of Reliability
Key Mitigation	Data Auditing & Cleaning	RLHF/ Red Teaming	Input Filtering/ Instruction Hierarchy	RAG/ Decoding Strategies

Table 1: Comparative Analysis of LLM Vulnerabilities. This matrix synthesizes the key differences in attack vectors, root causes, and mitigation strategies across the four categories.

4. CONCLUSION AND FUTURE DIRECTIONS

The security of Large Language Models is a complex and multi-faceted challenge that cannot be reduced to a single point of failure. This report has analyzed four primary categories of vulnerabilities; Jailbreaking, Prompt Injection, Data Poisoning, and Hallucination; and demonstrated that they are not isolated issues but interconnected parts of a larger systemic problem. From inference-time manipulations that exploit architectural flaws to training-time attacks that corrupt a model's core knowledge, each vulnerability reveals a different facet of the gap between the capabilities of LLMs and the requirements for safe, reliable deployment.

By organizing scattered academic works into a unified framework, this survey provides a clear, student-friendly guide for understanding the broader LLM security landscape. Highlighting the similarities, differences, and causal links between these threats helps developers and researchers see the bigger picture, moving beyond isolated fixes toward more comprehensive solutions. This holistic perspective is essential for building robust defenses and fostering a deeper understanding of the risks involved in deploying these powerful technologies.

Looking forward, the field must move beyond fixing risks one by one. Future research should focus on developing holistic security approaches that address the foundational architectural and data-related weaknesses highlighted throughout this survey. This includes creating models with inherent, hard-coded distinctions between instructions and data, developing verifiable and auditable data supply chains, and designing new alignment techniques that are more resilient to both adversarial attacks and inherent failures like hallucination. By adopting this unified perspective, the AI community can work toward building the next generation of LLMs that are not only more capable but also fundamentally more robust, reliable, and trustworthy.

5. CONTRIBUTIONS

Team member	Roles
Ketki	- Study and summarize literature on jailbreaks. - Study and summarize literature on data poisoning. - Take notes on methods, results, and defenses for these categories.
Lalitha	- Study and summarize literature on prompt injections. - Study and summarize literature on hallucinations. - Take notes on methods, results, and defenses for these categories.
Both	- Write the summary and discussion sections. - Identify relationships and differences across all four vulnerability types.

- Prepare tables, figures, and the final report draft.
--

Here is the **Categorized Bibliography** formatted for your report. I have filled in the missing authors, years, and specific conference details (e.g., USENIX Security, EMNLP, CCS) to make it professional.

You can simply **copy and paste** this entire section to the end of your report (as Section 6), and it will cover all your references without requiring you to change any text in the main body.

6. REVIEWED LITERATURE

1. **Universal & Transferable Attacks (GCG):** Zou, A., Wang, Z., Kolter, J. Z., & Fredrikson, M., "Universal and Transferable Adversarial Attacks on Aligned Language Models," *arXiv preprint*, 2023.
2. **Red Teaming LMs with LMs:** Perez, E., et al., "Red Teaming Language Models with Language Models," *EMNLP*, 2022.
3. **PAIR:** Chao, P., et al., "Jailbreaking Black Box Large Language Models in Twenty Queries," *NeurIPS*, 2023.
4. **TAP:** Mehrotra, A., et al., "Tree of Attacks: Jailbreaking Black-Box LLMs Automatically," *arXiv preprint*, 2023.
5. **AutoDAN:** Liu, X., et al., "AutoDAN: Interpretable Gradient-Based Adversarial Attacks on Large Language Models," *USENIX Security Symposium*, 2024.
6. **GPT-4 Is Too Smart To Be Safe (Cipher):** Yuan, Y., et al., "GPT-4 Is Too Smart To Be Safe: Stealthy Chat with LLMs via Cipher," *arXiv preprint*, 2023.
7. **CodeChameleon:** Lv, H., et al., "CodeChameleon: Personalized Encryption Framework for Jailbreaking Large Language Models," *arXiv preprint*, 2024.
8. **Weak-to-Strong Jailbreaking:** Zhao, X., et al., "Weak-to-Strong Jailbreaking on Large Language Models," *arXiv preprint*, 2024.
9. **Jailbreak in the Wild:** Shen, X., et al., "'Do Anything Now': Characterizing and Evaluating In-The-Wild Jailbreak Prompts," *ACM CCS*, 2024.
10. **JailbreakBench:** Chao, P., et al., "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," *arXiv preprint*, 2024.
11. **Automated Jailbreak Analysis:** Chen, et al., "Automated Jailbreak Analysis," *USENIX Security Symposium*, 2025.
12. **Jailbreaking via Prompt Engineering:** Deng, G., et al., "Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study," *EMNLP*, 2023.
13. **Multilingual Jailbreak Challenges:** Deng, Y., et al., "Multilingual Jailbreak Challenges in Large Language Models," *IJCNLP*, 2023.
14. **Persuasive Adversarial Prompts:** Zeng, Y., et al., "How Johnny Can Persuade LLMs to Jailbreak Them: Rethinking Persuasion to Challenge AI Safety," *NAACL*, 2024.

15. **SelfDefend:** Wang, X., et al., "SelfDefend: LLMs Can Defend Themselves against Jailbreaking in a Practical Manner," *USENIX Security Symposium*, 2025.
16. **Ignore Previous Prompt:** Greshake, K., et al., "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *ACM Workshop on Technology and Consumer Protection*, 2023.
17. **Compromising Safety via Direct Injection:** Wang, et al., "Compromising Safety via Direct Injection," *USENIX Security Symposium*, 2025.
18. **TopicAttack:** Chen, Y., et al., "TopicAttack: An Indirect Prompt Injection Attack via Topic Transition," *EMNLP Main Conference*, 2025 (Paper 372).
19. **Separating Instructions from Data:** *ICLR Conference Paper*, 2025 (e.g., "Can LLMs Separate Instructions From Data?").
20. **Jailbreak-Tuning:** Che, Z., et al., "Data Poisoning in LLMs: Jailbreak-Tuning and Scaling Laws," *arXiv preprint*, 2024.
21. **Persistent Pre-Training Poisoning:** Zhang, Y., et al., "Persistent Pre-Training Poisoning of LLMs," *ICLR*, 2025.
22. **Factuality of LLMs:** Wang, Y., et al., "Factuality of Large Language Models: A Survey," *EMNLP Main Conference*, 2024 (Paper 1088).
23. **TruthfulQA:** Lin, S., et al., "TruthfulQA: Measuring How Models Mimic Human Falsehoods," *ACL*, 2022.
24. **SelfCheckGPT:** Manakul, P., et al., "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection," *EMNLP*, 2023.